

Computing the conjugate of convex piecewise linear-quadratic bivariate functions

Bryan Gardiner · Yves Lucet

Received: 1 March 2011 / Accepted: 6 November 2011 / Published online: 22 March 2013
© Springer-Verlag Berlin Heidelberg and Mathematical Optimization Society 2013

Abstract We present a new algorithm to compute the Legendre–Fenchel conjugate of convex piecewise linear-quadratic (PLQ) bivariate functions. The algorithm stores a function using a (primal) planar arrangement. It then computes the (dual) arrangement associated with the conjugate by looping through vertices, edges, and faces in the primal arrangement and building associated dual vertices, edges, and faces. Using optimal computational geometry data structures, the algorithm has a linear time worst-case complexity. We present the algorithm, and illustrate it with numerical examples. We proceed to build a toolbox for convex bivariate PLQ functions by implementing the addition, and scalar multiplication operations. Finally, we compose these operators to compute classical convex analysis operators such as the Moreau envelope, and the proximal average.

Keywords Computer-aided convex analysis · Computational convex analysis · Fenchel conjugate · Legendre–Fenchel transform · Convex subdifferential · Planar arrangement

Yves Lucet was partially supported by a Discovery grant from the Natural Sciences and Engineering Research Council of Canada (NSERC). Bryan Gardiner was partially supported by an NSERC Undergraduate Student Research Award. Part of the research was done in the Computer-Aided Convex Analysis laboratory funded by the Canadian Foundation for Innovation under a Leaders Opportunity Fund.

B. Gardiner · Y. Lucet (✉)

Computer Science, I. K. Barber School, ASC 350, University of British Columbia Okanagan,
Kelowna, BC, V1V 1V7, Canada
e-mail: yves.lucet@ubc.ca

B. Gardiner
e-mail: khumba@interchange.ubc.ca

Mathematics Subject Classification (2000) 90C25 · 26A51 · 26B25 · 47H05 · 52A41

1 Introduction

Computational convex analysis focuses on the computation of operators frequently encountered in convex analysis, like the (Legendre–Fenchel) conjugate, the Moreau envelope (also called Moreau–Yosida approximate or Moreau–Yosida regularization), and their combinations, like the proximal average. Its origin can be traced back to the work of Moreau [29, Paragraph 5c], although it was formally born with the introduction of a fast algorithm, named the fast Legendre transform, to compute the conjugate [9, 12, 21, 30, 33]. Applications of computational convex analysis are not limited to computer-aided convex analysis as pointed out by the recent survey [26].

Symbolic computation algorithms have been built to compute the conjugate. Univariate [6] and multi-variate [7] functions can be handled with these packages. However, without restricting the computation to a specific class of functions, symbolic computation is limited to the cases for which one can invert the gradient explicitly, which is not always possible even for sixth-degree polynomials. Hence, the need for numerical algorithms.

The fast Legendre transform and (its improvement) the linear-time Legendre transform [23] algorithms use a piecewise linear model and can be applied theoretically to functions of any number of variables. Of course, the curse of dimensionality (capturing the graph of a function requires a number of points that grows exponentially with the dimension) restricts their applications to functions with both a low number of variables and a bounded domain. Nevertheless a toolbox for convex operators can be built based on a conjugate algorithm. For example, the Moreau envelope can be computed from the conjugate. Adding an algorithm to compute the convex envelope of a function [15], one obtains a fairly complete toolbox to compute most operators encountered in convex analysis applied to convex or nonconvex functions. Indeed, the required core operators are addition, scalar multiplication, and conjugacy for a convex toolbox, while a nonconvex toolbox only requires adding the convex envelope operation. (Optional operators like the maximum or the minimum of two functions can also be added.) Then other operators can be computed by composition e.g. the Moreau envelope and the proximal average. The Computational Convex Analysis (CCA) toolbox implements many algorithms like the fast Legendre transform, and the linear-time Legendre transform. The toolbox is freely available for download at [22].

Algorithms based on a first order model [21, 23, 25] were found ineffective to compute operators like the proximal average (defined in [4] and studied in [1–3, 5, 20, 28]). Indeed, any operator that requires the composition of several convex operators with unbounded domains requires much technical expertise to control the resulting numerical errors [25]. As a remedy, algorithms based on piecewise linear-quadratic (PLQ) functions were introduced [27]. These PLQ algorithms only apply to univariate functions. Their performance was subsequently improved for convex univariate PLQ functions by graph matrix calculus algorithms [16] (also part of the CCA toolbox).

To our knowledge there is no effective algorithm to compute the conjugate of bivariate PLQ functions as all previous work dealt with (multivariate) piecewise linear functions, univariate PLQ functions, or symbolic computation. We present the first such algorithm for convex PLQ functions in the present paper. It relies on explicitly storing a planar arrangement with the associated quadratic functions. Evaluating the value of the function requires first to locate the right “face” by applying a computational geometry algorithm for the point location problem [14, Chapter 6]. We also implement the necessary operators to build a toolbox for convex PLQ functions: the addition operation, which is called the map overlay problem in computational geometry and is well-known to take more than linear-time [14, Chapter 2]), and scalar multiplication, which is trivially linear-time. Hence, we obtain a toolbox for working with convex bivariate PLQ functions, which is optimal for worst-case time complexity.

The paper is organized as follows. The present section introduces the paper; Sect. 2 fixes the notations and recall several convex analysis results. Section 3 presents the algorithm while Sect. 4 illustrates it with several examples. Then Sect. 5 discusses how to build a toolbox, including the performance of algorithms and the difference with the univariate case. Finally, Sect. 6 summarizes our results and proposes future directions.

2 Preliminaries

First, we fix our notations. We consider functions $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$. The set of points where the function takes finite values is called the effective domain and denoted $\text{dom } f$ i.e. $\text{dom } f = \{x \in \mathbb{R}^2 : f(x) < +\infty\}$. A function f is proper if $\text{dom } f$ is nonempty. Unless otherwise stated all functions considered are proper lower semicontinuous (lsc) convex functions. We denote by $\text{ri} S$ the relative interior of a set S , $\|\cdot\|$ the Euclidean norm, and $\langle \cdot, \cdot \rangle$ the standard dot product for which we also use the matrix multiplication notation $x^T y = \langle x, y \rangle$. Moreover, we note $\square$ the inf-convolution operator defined by

$$f \square g(x) = \inf_{y \in \mathbb{R}^2} [f(y) + g(x - y)],$$

while

$$\partial f(x) = \{s \in \mathbb{R}^2 : \forall y \in \mathbb{R}^2, f(y) \geq f(x) + \langle s, y - x \rangle\}$$

denotes the convex subdifferential. We write the (Legendre–Fenchel) conjugate as

$$f^*(s) = \sup_{x \in \mathbb{R}^2} [\langle s, x \rangle - f(x)],$$

and the Moreau(–Yosida) envelope of a function f with parameter $\lambda > 0$ as

$$e_\lambda f(x) = (f \square \lambda^{-1} q)(x) = \min_{y \in \mathbb{R}^2} \left[f(y) + \frac{\|x - y\|^2}{2\lambda} \right]$$

where q is the quadratic kernel $q(x) = \|x\|^2/2$. The Moreau envelope (resp. conjugate) can then be easily deduced from the conjugate (resp. Moreau envelope) by using the following formulas (see [24] or prove directly)

$$e_\lambda f(x) = \frac{\|x\|^2}{2\lambda} - \frac{1}{\lambda} \left(\frac{\|\cdot\|^2}{2} + \lambda f \right)^*(x), \quad (1)$$

$$f^*(s) = \frac{\|s\|^2}{2} - \lambda e_\lambda \left(\frac{1}{\lambda} f - \frac{\|\cdot\|^2}{2\lambda} \right)(s), \quad (2)$$

where $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$, and $\lambda > 0$. We recall the self-dual smoothing operator

$$s_\lambda f = (1 - \lambda^2)e_\lambda f + \lambda q,$$

introduced in [17], and another self-dual operator

$$T_\lambda f = \mathcal{P}(f, q; 1 - \lambda, \lambda)$$

introduced in [5, 28], which uses the proximal average defined next.

The *proximal average* is the function $\mathcal{P}_\mu(f_1, f_2; \lambda_1, \lambda_2) : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ defined by

$$\mathcal{P}_\mu(f_1, f_2; \lambda_1, \lambda_2) = \left(\lambda_0 \left(f_0 + \frac{1}{\mu} q \right)^* + \lambda_1 \left(f_1 + \frac{1}{\mu} q \right)^* \right)^* - \frac{1}{\mu} q,$$

where the functions $f_0, f_1 : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ are proper lsc convex functions, $\mu > 0$, and $\lambda_0, \lambda_1 > 0$ with $\lambda_0 + \lambda_1 = 1$. When there is no ambiguity on $f_0, f_1, \lambda_0, \lambda_1$, we will simplify the notation to $\mathcal{P}$. The proximal average is a very useful convex operator as it is a convex function [1, Corollary 5.2] that satisfies $\text{dom } \mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) = \lambda_0 \text{dom } f_0 + \lambda_1 \text{dom } f_1$ [1, Theorem 4.6]. Moreover, the proximal mapping of the proximal average is the convex combination of the proximal mappings: $\text{Prox } \mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) = \lambda_0 \text{Prox } f_0 + \lambda_1 \text{Prox } f_1$ [1, Theorem 6.7], and the conjugate of the proximal average is the proximal average of the conjugate [1, Theorem 5.1] $[\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1)]^* = \mathcal{P}(f_0^*, f_1^*; \lambda_0, \lambda_1)$.

We will work with (piecewise linear-quadratic) PLQ functions, which were described in [31] and implemented in [27]. For univariate functions, they are extended-valued piecewise quadratic functions defined on a finite number of intervals which are continuous on the interior of their effective domain. In other words, for any $x \in \mathbb{R}$ a PLQ function f is defined by a partition of $\mathbb{R}$ into intervals $[x_i, x_{i+1}]$ such that when $x \in [x_i, x_{i+1}]$ we have $f(x) = a_i x^2 + b_i x + c$ where $a, b \in \mathbb{R}$ and $c \in \mathbb{R} \cup \{+\infty\}$. (A univariate PLQ function is a quadratic spline).

To define them precisely for bivariate functions we first define a *vertex* to be a point in the plane, an *edge* $E = \{x \in \mathbb{R}^2 : x = \lambda_1 x^1 + \lambda_2 x^2\}$ with $x^1 \neq x^2$ and $\lambda_1 + \lambda_2 = 1$ to be either a (bounded) segment when $\lambda_1 \geq 0, \lambda_2 \geq 0$, a ray or half-line when $\lambda_1 \geq 0$ but $\lambda_2 \in \mathbb{R}$, or a line in the plane when $\lambda_1, \lambda_2 \in \mathbb{R}$. We recall the definition of a polyhedral set.

Definition 1 ([31, Example 2.10]) (Polyhedral sets and affine sets). A set $C \subset \mathbb{R}^2$ is said to be a polyhedral set if it can be expressed as the intersection of a finite family of closed half-spaces or hyperplanes, or equivalently, can be specified by finitely many linear constraints, i.e., constraints $f_i(x) \leq 0$ or $f_i(x) = 0$ where f_i is affine. It is called an affine set if it can be expressed as the intersection of hyperplanes alone, i.e., in terms only of constraints $f_i(x) = 0$ with f_i affine.

Affine sets are in particular polyhedral, while polyhedral sets are in particular closed, convex sets. The empty set and the whole space are affine.

Polyhedral sets with nonempty interior in the plane will be called *faces*.

We can now define convex PLQ bivariate functions.

Definition 2 A convex PLQ bivariate function is a convex function $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ whose domain can be partitioned as a finite number of polyhedral sets P_i on each of which the function is quadratic i.e. for $x \in P_i$, $f(x) = 1/2x^T A_i x + b_i^T x + c_i$ with A_i a 2×2 symmetric positive semi-definite matrix, $b_i \in \mathbb{R}^2$, and $c_i \in \mathbb{R} \cup \{+\infty\}$.

(All vectors are column vectors, and the notation x^T denotes the transpose of x i.e. the row vector $[x_1, x_2]$.) This definition of PLQ functions follows [31, Definition 10.20 p. 440].

As examples, the function $f(x_1, x_2) = |x_1| + |x_2|$ is convex PLQ since its domain $\mathbb{R}^2$ can be partitioned into the four quadrants which are polyhedral sets, and it is linear on each piece. However the function $f(x_1, x_2) = |x_1^2 + x_2^2 - 1|$ is not PLQ: its domain can not be partitioned as a finite number of polyhedral sets since $C_1 = \{(x_1, x_2) : x_1^2 + x_2^2 - 1 \leq 1\}$ and $C_2 = \{(x_1, x_2) : x_1^2 + x_2^2 - 1 \geq 1\}$ are not polyhedral sets [31, p. 440]. The latter example also shows that the maximum of two PLQ functions is not necessarily a PLQ function when the function has more than one variable.

Fact 1 ([31, Proposition 10.21]) Assume $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ is a proper convex PLQ bivariate function. Then $\text{dom } f$ is closed, f is continuous on $\text{ri dom } f$, lsc on $\mathbb{R}^2$, and $\partial f(x)$ is a nonempty polyhedral set at any point $x \in \text{dom } f$.

The reason we are interested in PLQ functions is that the conjugate of a PLQ function is a PLQ function [31, Theorem 11.14]. In fact, the class of PLQ functions is closed under most convex operators, see [31, Exercise 10.22, Example 11.28, Proposition 11.32].

Fact 2 Assume $f, f_1, f_2 : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ are proper convex PLQ functions, and $\lambda > 0$.

1. (Addition) The sum $f_1 + f_2$ is proper convex PLQ.
2. If $g(x) = f(Ax + a)$, then g is proper convex PLQ.
3. If $g(x) = f(x, \bar{u})$, then g is proper convex PLQ.
4. (Moreau envelope) The Moreau envelope $e_\lambda f$ is proper convex PLQ.
5. (Conjugate) The conjugate f^* is proper convex PLQ.
6. (Scalar multiplication) The scalar multiple λf is proper convex PLQ. The scalar multiplication by $\lambda < 0$ of a proper PLQ function is still a proper PLQ function.
7. (Inf-convolution) The function $f = f_1 \square f_2$ is proper, convex PLQ, unless f is improper with $\text{dom } f$ a polyhedral set on which $f = -\infty$. Either way, f is lsc.

8. (Image of a function under a linear mapping) If for a matrix $A \in \mathbb{R}^{m \times n}$ the function $g(u) = (Af)(u) = \inf_x \{f(x) : Ax = u\}$ then g is proper convex PLQ on $\mathbb{R}^m$, unless g is improper with $\text{dom } g$ a polyhedral set on which $g = -\infty$. Either way, g is lsc.
9. (Marginal function) If $p(u) = \inf_x f(x, u)$ then p is proper convex PLQ unless p is improper with $\text{dom } p$ a polyhedral set on which $p = -\infty$. Either way, p is lsc.
10. (Proximal average) If $g(x) = \mathcal{P}_\mu(f_1, f_2; 1 - \lambda, \lambda)$ then g is proper convex PLQ.

Consequently, we can approximate a given function numerically by a PLQ function and then perform all computations within the class of PLQ functions. In addition, PLQ functions completely capture the domain information so contrary to piecewise linear functions we are not practically restricted to functions with bounded domain.

We now recall a few results from convex analysis.

Fact 3 ([19, X Example 1.1.4], [31, Example 11.10] Convex degenerate quadratic) Consider the function $f(x) = \frac{1}{2}x^T Qx + b^T x$ for $x \in \mathbb{R}^2$ where Q is a 2×2 symmetric positive semi-definite matrix. Then

$$f^*(s) = \frac{1}{2}(s - b)^T Q^-(s - b) + I_{b + \text{Im } Q}(s) \quad (3)$$

where Q^- denotes the Moore-Penrose pseudo-inverse of Q , $\text{Im } Q = \{y \in \mathbb{R}^2 : \exists x \in \mathbb{R}^2, y = Qx\}$ is the image of Q , and I_S denotes the indicator function of the set S : $I_S(s) = 0$ if $s \in S$ and $+\infty$ otherwise.

When the function f is not defined everywhere, we use [19, X Proposition 1.3.2] and [19, X Proposition 1.3.4] to compute the conjugate defined on an affine subspace. Finally, the following result is well-known but we recall it as it is the foundation of our algorithm.

Proposition 1 ([19, XI proposition 1.2.1]) Assume f is a proper lsc convex function. The vector $s \in \mathbb{R}^2$ is a subgradient of f at $x \in \text{dom } f$ if, and only if,

$$f^*(s) + f(x) = s^T x.$$

As a result,

$$s \in \partial f(x) \iff x \in \partial f^*(s).$$

3 The algorithm

We name *entity* a set in $\mathbb{R}^2$ which is either a vertex, an edge, or a face.

The basic idea of the algorithm is to loop through each entity of the domain of f and compute its associated dual entity. We store the decomposition of the domain of a function in a planar arrangement data structure provided by CGLAB [11, Chapter 30]. Then we assign to each entity an index that is associated with a quadratic function.

To evaluate the function f at a point $x \in \mathbb{R}^2$, we first use a planar point-location algorithm [11, Section 30.3.1] to find the entity to which x belongs, then we evaluate the

associated quadratic function. Hence, evaluating a function containing $O(n)$ entities takes $O(\log n)$ time in the worst-case.

The addition of two PLQ functions requires computing the overlay of the two planar arrangements. Then we loop through each entity in the overlay and add the associated quadratic functions. The scalar multiplication operation is even simpler as the planar arrangement is unmodified; only the quadratic functions are multiplied. The same is true when we add a multiple of the norm squared to a PLQ function.

Unfortunately, computing the conjugate is much more involved. We will use the following additional notations. A QQ (resp. QL, LL) face is a face associated with a quadratic function with a symmetric positive definite (resp. positive semi-definite, zero) matrix. From now on we assume f is a convex PLQ bivariate function.

Using Proposition 1, the computation of the conjugate f^* at s amounts to finding an x with $s \in \partial f(x)$. Such an x necessarily lies in a primal entity (since the conjugate is a PLQ function) and any other s' in the same dual entity as s will have an x' lying in the same primal entity. In other words, for each primal entity E we are looking for the set of points

$$S_E = \{s \in \mathbb{R}^2 : \exists x \in E, f^*(s) = s^T x - f(x)\},$$

which is exactly the set $\partial f(E)$ (by Proposition 1), and forms an entity in $\text{dom } f^*$. As the entities E form a partition of $\mathbb{R}^2 \supset \text{dom } f$, the corresponding entities $S_E = \partial f(E)$ form a partition of the dual space $\mathbb{R}^2 \supset \text{dom } f^*$. (To compute $\partial f(E)$, we use Proposition 1 i.e. we compute ∂f , switch variables to obtain $(\partial f)^{-1}$ and deduce f^*).

Remark 1 To reduce computations, we note that a partition is completely defined by its edges since vertices can be found by intersecting edges and faces are defined by their boundaries. Of course, edges are defined from vertices. Consequently, we only need to compute the vertices of $\text{dom } f^*$ and define its edges to completely define the arrangement representing $\text{dom } f^*$. (The case when the primal arrangement does not contain any edge is treated as a special case).

For example, Fig. 1a shows the domain of a PLQ function and its associated quadratic functions. Its conjugate is shown in Fig. 1b. Note that the primal vertex $(0, 0)$ is associated with the dual face $\{s = (s_1, s_2) \in \mathbb{R}^2 : -2 \leq s_1, s_2 \leq 0\}$, the ray $\{x = (x_1, x_2) \in \mathbb{R}^2 : x_1 \leq 0, x_2 = 0\}$ with the segment $\{s : s_1 = -2, -2 \leq s_2 \leq 0\}$, and the ray $\{x : x_1 = 0, x_2 \geq 0\}$ with the face $\{s : -2 \leq s_1 \leq 0, 0 \leq s_2\}$. Apart from the QQ faces that are obviously conjugate to each other, finding the associated entities is not straightforward.

The algorithm is summarized in Algorithm 1. It first handles special cases i.e. domains with no edge entity: point indicator functions (conjugate is a single LL face), single QQ face (conjugate is a single QQ face), single QL face (conjugate is a quadratic function defined on a line), and a single LL face (conjugate is a point indicator function) on Line 1. It then calculates the conjugate on all the edges on Line 2, and merges them to build the domain of the conjugate on Line 3. Finally, it assigns each entity with its quadratic function on Line 4.

To compute the conjugate of an edge we consider whether the function is linear or quadratic (i.e. has a nonzero quadratic term) along the edge, whether the gradient of

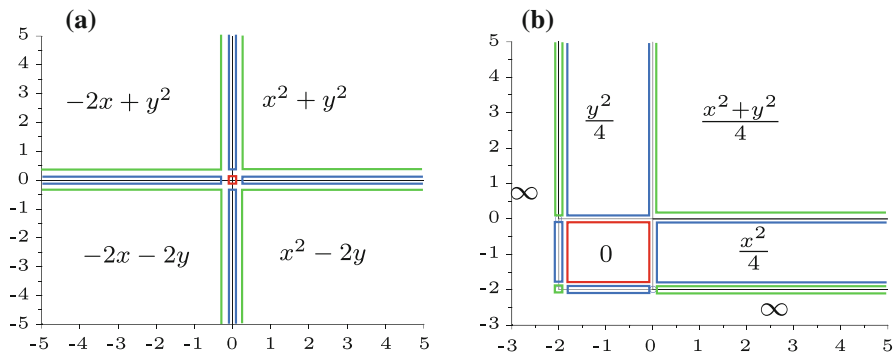


Fig. 1 A convex PLQ function (*left*) and its conjugate (*right*). The *colors* show the associated entities. **a** Finite convex PLQ function f with four faces (one QQ, two QL, and one LL), four edges (two linear edges and two quadratic edges), and one vertex. **b** Its conjugate f^* is a convex PLQ function with four faces (one QQ, two QL, and one LL), eight edges (four quadratic, and five linear), and four vertices (color figure online)

Algorithm 1: Conjugate Computation Algorithm

Input: A : arrangement, F : list of associated quadratic functions

Output: A^* : arrangement, F^* : list of associated quadratic functions

begin

- 1 Handle special cases: Point indicator, single QQ, QL, LL face;
- 2 Calculate the conjugate on all edges;
- 3 Merge into a single conjugate domain model;
- 4 Assign function values to conjugate entities;

end

the incident faces agree (smooth or kinked case), and the boundedness of the edge (is the edge a segment, an half line, or a line?).

Let us first show how the algorithm works through an example. Consider the function

$$f(x_1, x_2) = |x_2| + I_{[-1,1] \times [-1,1]}(x_1, x_2).$$

Figure 2a (resp. Fig. 2b) shows the primal (resp. dual) domain. It contains 7 linear edges. The algorithm will consider each one in turn and map them according to Table 1. Putting all the edges together, we add the faces to obtain the arrangement storing the domain of the conjugate. Finally, we compute the value of the function associated with each entity.

Computing the dual entity associated with an edge is the key to the algorithm. We consider a horizontal edge as in Fig. 3 with endpoints x_A (on the left), x_B (on the right). (If the edge is vertical, rotate it by $\pi/2$ to deduce which case applies.) One or both endpoints could be at infinity to model a ray or a line. The face F_i (resp. F_j) is associated with a quadratic function $f_i(x)$ (resp. $f_j(x)$). We distinguish two cases: the function associated with the edge is linear or it is quadratic (i.e. has a nonzero quadratic term).

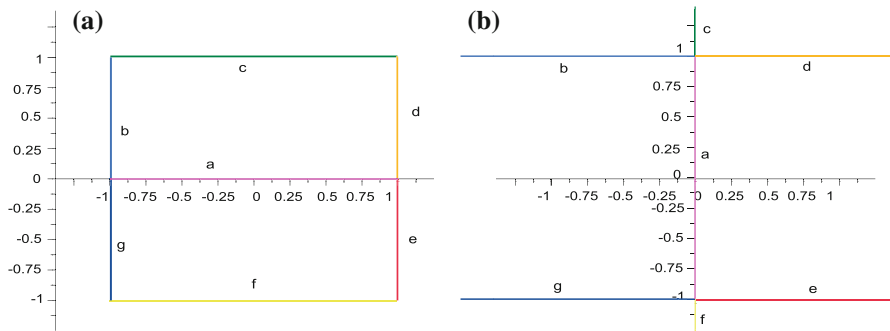
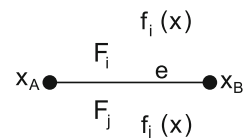


Fig. 2 The primal (a) and dual domains (b) of the function $f(x_1, x_2) = |x_2| + I_{[-1,1] \times [-1,1]}(x)$

Table 1 Mapping of primal edges to dual edges

Number	Primal edge	Dual edge
a	$[-1, 1] \times \{0\}$	$\{0\} \times [-1, 1]$
b	$\{-1\} \times [0, 1]$	$(-\infty, 0] \times \{1\}$
c	$[-1, 1] \times \{1\}$	$\{0\} \times [1, +\infty)$
d	$\{1\} \times [0, 1]$	$[0, +\infty) \times \{1\}$
e	$\{1\} \times [-1, 0]$	$[0, +\infty) \times \{-1\}$
f	$[-1, 1] \times \{-1\}$	$\{0\} \times (-\infty, -1]$
g	$\{-1\} \times [-1, 0]$	$(-\infty, 0] \times \{-1\}$

Fig. 3 Sample edge e with endpoints x_A, x_B (not necessarily bounded), neighboring faces F_i, F_j with associated quadratic functions $f_i(x)$, and $f_j(x)$



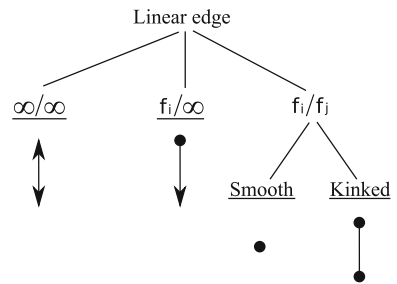
3.1 Linear edges

When the function associated with an edge is linear, we call the edge a *linear edge*. Using the notation of Fig. 3, we will say a linear edge is *smooth* if $\nabla f_i(x_A) = \nabla f_j(x_A)$. Otherwise, we say the linear edge is *kinked*. (Since the edge is linear we always have $\nabla f_i(x_A) = \nabla f_i(x_B)$ and $\nabla f_j(x_A) = \nabla f_j(x_B)$).

Four cases must be considered to compute its dual entity as summarized in Fig. 4. Reading the tree from left to right we have the following cases.

1. The left-most branch of the tree symbolizes the case where both neighboring faces are associated with the $+\infty$ value. In that case, the conjugate is linear in a direction perpendicular to the edge, and $+\infty$ everywhere else. We symbolize that with a vertical line. For example, consider the function $f(x_1, x_2) = I_{[-1,1]}(x_1) + I_{\{0\}}(x_2)$ whose conjugate is $f^*(s_1, s_2) = |s_1| + 0$. More generally, the function $f(x_1, x_2) = ax_1 + b + I_{\{0\}}(x_2)$ admits for conjugate $f^*(s_1, s_2) = I_{\{a\}}(s_1) - b + 0$.
2. The second case occurs when a face F_i is associated a finite value while the other face F_j is associated with $+\infty$. For example consider the function $f(x_1, x_2) =$

Fig. 4 The associated 4 cases depending on whether a face is associated with ∞ and whether the edge is smooth or kinked



$\frac{1}{2}x_2^2 + I_{\{x_2: x_2 \leq 0\}}(x_2)$. Then its conjugate is $f^*(s_1, s_2) = \frac{1}{2}s_2^2 + I_{\{s_2: s_2 \geq 0\}}(s_2) + I_{\{0\}}(s_1)$. Hence, the edge is transformed into a ray.

3. The next two cases involve an edge with two faces associated with finite values.
 - When the edge is smooth, the entity dual to the edge is a vertex e.g. $f(x_1, x_2) = \frac{1}{2}x_2^2$ when $x_2 \leq 0$, and $f(x_1, x_2) = x_2^2$ when $x_2 \geq 0$ has conjugate $f^*(s_1, s_2) = \frac{1}{2}s_2^2 + I_{\{0\}}(s_1)$ when $s_2 \leq 0$ and $f^*(s_1, s_2) = 2s_2^2 + I_{\{0\}}(s_1)$ otherwise.
 - When the edge is kinked, the entity dual to the edge is a segment e.g. $f(x_1, x_2) = \frac{1}{2}x_2^2 + x_2$ when $x_2 \leq 0$, and $f(x_1, x_2) = \frac{1}{2}x_2^2 - x_2$ when $x_2 \geq 0$ has conjugate $f^*(s_1, s_2) = \frac{1}{2}s_2^2 - s_2 + \frac{1}{2}$ when $s_2 \leq 0$ and $f^*(s_1, s_2) = \frac{1}{2}s_2^2 + s_2 + \frac{1}{2}$ otherwise.

3.2 Quadratic edges

For computing the dual of an edge associated with a quadratic nonlinear function (that we denote a *quadratic edge*), 15 cases occur as illustrated in Fig. 5. Similarly to linear edges, we say an edge is *kinked* at point x_A (or kinked on the left) when $\nabla f_i(x_A) \neq \nabla f_j(x_A)$ and *smooth* (on the left) otherwise.

We split these cases into 3 main subcases as follow. (The notation A means the gradient of neighborhood quadratic functions are equal at the point x_A i.e. $\nabla f_i(x_A) = \nabla f_j(x_A)$; similarly, the notation A' means $\nabla f_i(x_A) \neq \nabla f_j(x_A)$).

1. Both neighborhood faces are associated with $+\infty$. Then the entity dual to the quadratic edge is
 - a face bounded by two lines, when the quadratic edge is a segment,
 - a face bounded by a line when the quadratic edge is a ray, and
 - a QL face when the quadratic edge is a line.
2. Similarly when only one neighborhood face has finite values, the entity dual to the edge is
 - a face bounded by 2 rays and a segment when the quadratic edge is a segment,
 - a face bounded by 2 rays when the quadratic edge is a ray, and
 - a face bounded by a line when the quadratic edge is a line.
3. When both faces have finite values we have the same 3 subcases:
 - when the quadratic edge is a segment we further distinguish 4 cases:
 - $[A \wedge B]$ when the edge at both endpoints is smooth, we obtain a segment,

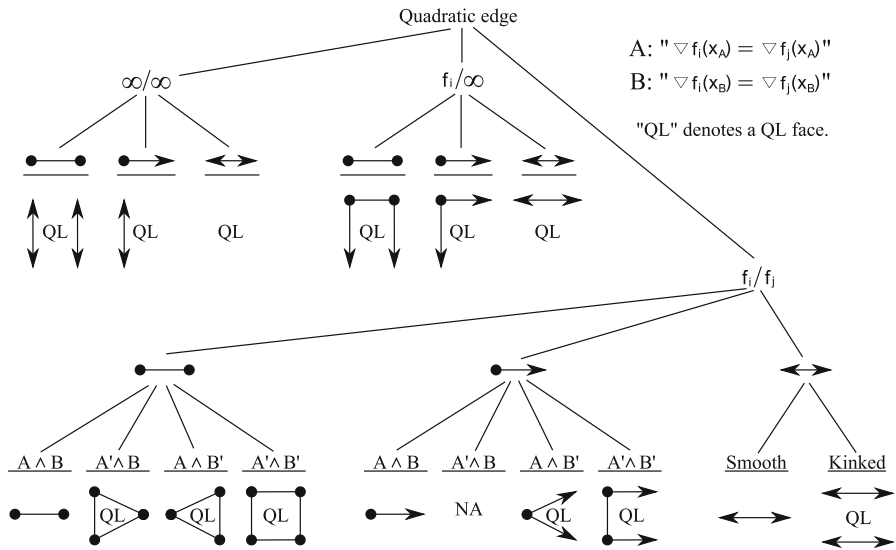


Fig. 5 The 15 cases associated with a quadratic edge depend on whether a face is associated with ∞ , whether the edge is a segment, ray, or line, and whether the edge is smooth or kinked. (NA means the case does not occur as it gives rise to a nonconvex function.)

- $[A' \wedge B]$ when the edge at the left endpoint is kinked but the edge at the right endpoint is smooth, we get a triangular face,
- $[A \wedge B']$ symmetric case of the previous one,
- $[A' \wedge B']$ when both edges are kinked, we obtain a square face.
- when the quadratic edge is a ray, only 3 of the 4 subcases can occur. Then the entity dual to the edge is
 - $[A \wedge B]$ a ray when we have smooth edges at the vertices,
 - $[A' \wedge B]$ a nonconvex function when we have a kinked edge at the left vertex and a smoothed edge at the right vertex, hence that case cannot happen.
 - $[A \wedge B']$ a face bounded by 2 rays and a segment when we have a smoothed edge on the left vertex and kinked one on the right vertex.
 - $[A' \wedge B']$ a face bounded by a segment and 2 rays when we have kinked edges at both vertices.
- when the quadratic edge is a line, we have 2 cases (the cases $A' \wedge B$ and $A \wedge B'$ do not result in a convex function, see Remark 2) and the entity dual to the edge is
 - $[A \wedge B]$ a line when both edges are smooth, while it is
 - $[A' \wedge B']$ a face bounded by 2 lines when both edges are kinked.

Before proving that a quadratic edge which is a ray cannot result in a convex function, we consider the following bivariate function

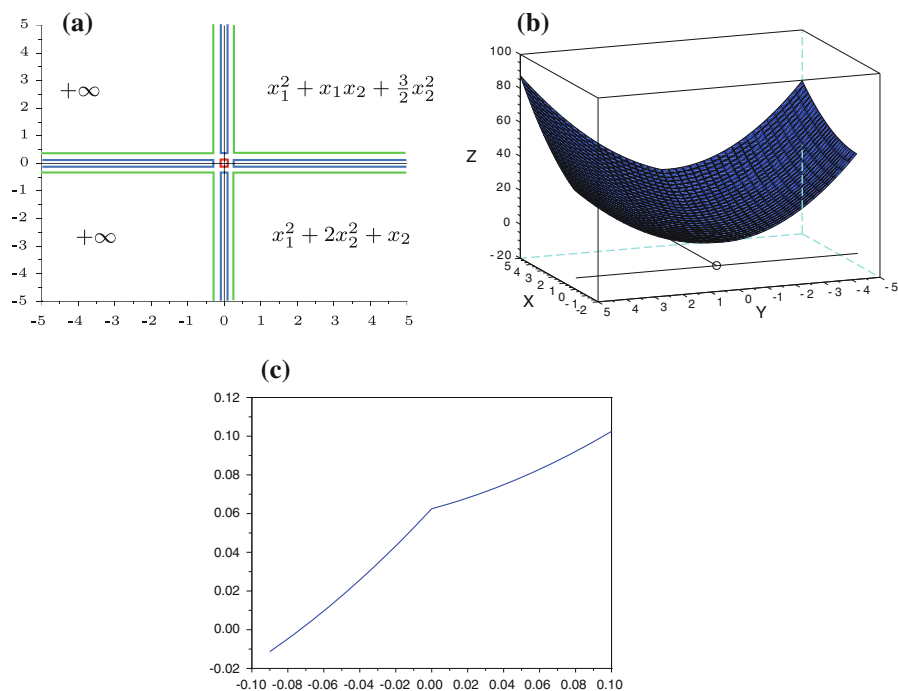


Fig. 6 The PLQ function f defined by (4) illustrating the nonconvexity of the $A' \wedge B$ case for a quadratic ray even though the function is continuous on its domain. **a** The primal domain. **b** The function f . **c** The slice $g(t) = f(1/4, t)$ illustrating the nonconvexity of the function f

$$f(x_1, x_2) = \begin{cases} x_1^2 + x_1x_2 + \frac{3}{2}x_2^2 & \text{if } x_1 \geq 1 \text{ and } x_2 \geq 0, \\ x_1^2 + 2x_2^2 + x_2 & \text{if } x_1 \geq 1 \text{ and } x_2 \leq 0, \\ +\infty & \text{otherwise.} \end{cases} \quad (4)$$

The domain of f is made of the quadratic ray $\{(x_1, x_2) \in \mathbb{R}^2 : x_1 = 0, x_2 > 0\}$ (the function equals $+\infty$ when $x_1 < 0$), the quadratic ray $\{(x_1, x_2) \in \mathbb{R}^2 : x_1 = 0, x_2 < 0\}$, the vertex $(0, 0)$, the ray $\{(x_1, x_2) \in \mathbb{R}^2 : x_1 \geq 0, x_2 = 0\}$, the QQ face $\{(x_1, x_2) \in \mathbb{R}^2 : x_1 > 0, x_2 > 0\}$, and the QQ face $\{(x_1, x_2) \in \mathbb{R}^2 : x_1 > 0, x_2 < 0\}$ (see Fig. 6).

While the function is proper PLQ bivariate, it is not convex. Intuitively, if the function f is convex the derivative of any function $g(t) = f(x + td)$ has to be nondecreasing. However, the y component of the gradient $\nabla f(x)$ is a ray for $x_2 > 0$ and another ray for $x_2 < 0$. Hence, convexity requires one ray to be above the other. But that condition is violated by requiring a kinked edge at $A = (0, 0)$ (both lines are distinct) and a smooth edge at $B = (1, 0)$ (both lines should coincide). More formally, consider $x = (0.25, 0.1)$, $y = (0.25, -0.1)$, and $\lambda = 0.5$. Then

$$f(\lambda x + (1 - \lambda)y) - \lambda f(x) - (1 - \lambda)f(y) = 0.02 > 0$$

Consequently the function is not convex as illustrated on Fig. 6c.

The following lemma generalizes the argument.

Lemma 1 Assume f is a proper (continuous) PLQ bivariate function having a quadratic ray with finite valued neighboring faces associated with quadratic functions $f_i(x) = 1/2x^T A_i x + b_i^T x + c_i$ that satisfy $\nabla f_1(x_A) \neq \nabla f_2(x_A)$ at the origin of the ray x_A , and $\nabla f_1(x_B) = \nabla f_2(x_B)$ at a point on the ray x_B . Then f is not convex.

Proof Without loss of generality we can assume $x_A = (0, 0)$, $x_B = (1, 0)$ (otherwise apply a translation, a rotation and a scaling). Note $A_i = \begin{bmatrix} a_{11}^i & a_{12}^i \\ a_{21}^i & a_{22}^i \end{bmatrix}$ where $a_{12}^i = a_{21}^i$.

Continuity of f implies $a_{11}^1 = a_{11}^2$, $b_1^1 = b_1^2$, and $c_1 = c_2$ while $\nabla f_1(x_B) = \nabla f_2(x_B)$ gives

$$a_{21}^1 + b_2^1 = a_{21}^2 + b_2^2, \quad (5)$$

and $\nabla f_1(x_A) \neq \nabla f_2(x_A)$ gives $b_2^1 \neq b_2^2$. The later with (5) implies $a_{12}^1 \neq a_{12}^2$.

Assume f is convex. Then $g(t) = f(\bar{x} + td)$ is convex univariate and g admits left $g'_-(t)$ and right $g'_+(t)$ derivatives with $g'_-(t) \leq g'_+(t)$. The function g is differentiable for $t \neq 0$ with $g'(t) = \nabla f(x + td)^T d$. Take $d = (0, -2)$, $\bar{x} = (\bar{x}_1, 0)$, and write $g'_-(0) \leq g'_+(0)$ to obtain

$$a_{12}^1 \bar{x}_1 + b_2^1 \geq a_{12}^2 \bar{x}_1 + b_2^2 \quad (6)$$

for all $\bar{x}_1 \geq 0$. Now for $\bar{x}_1 = 0$ we get $b_2^1 \geq b_2^2$ but since $b_2^1 \neq b_2^2$ we have $b_2^1 > b_2^2$. Inequality (6) also implies $a_{12}^1 \geq a_{12}^2$ (otherwise the inequality is violated for large values of $\bar{x}_1$) and since $a_{12}^1 \neq a_{12}^2$ we deduce $a_{12}^1 > a_{12}^2$.

Finally, adding $b_2^1 > b_2^2$ with $a_{12}^1 > a_{12}^2$ gives $a_{21}^1 + b_2^1 > a_{21}^2 + b_2^2$, which contradicts (5). $\square$

Remark 2 Lemma 1 can trivially be extended to show that we cannot have a quadratic line with either $A' \wedge B$ or $A \wedge B'$.

Remark 3 Additional results on the structure of convex PLQ functions are contained in [34] in which convex PLQ functions are further split between difference definite (the Hessian of the difference of two neighboring quadratic functions is positive or negative semi-definite) and difference indefinite (the Hessian of the difference is neither positive nor negative semi-definite) types. Convex PLQ functions of difference definite type defined on boxes are easily identified, and can be simplified as the sum of a convex quadratic function and a separable convex PLQ function, making their manipulation much simpler. We do not make such distinction and consider all convex PLQ functions.

3.3 Assigning function values to dual entities

We perform Step 4 of the algorithm as follow.

We first consider a primal vertex x_0 whose dual entity is a (LL) face. Such a face is assigned the function $f^*(s) = s^T x_0 - f(x_0)$.

Then we consider edges whose dual entity is a face. For each primal edge, we assign the conjugate function to its dual edge (if the edge is linear with function value $f(x) = x^T Ax + b^T x + c$ and x_0 is a point on the edge we assign $f^*(s) = x_0^T s - b^T x_0 - x_0^T Ax_0 - c$, if the edge is quadratic we compute the projection operator and use Formula (3)). The dual face is assigned conjugate functions based on the same formula.

Finally, we handle faces conjugating into faces using Formula (3).

3.4 Correctness and complexity

The justification of the algorithm is tedious but elementary. Since entities can only be vertices, edges, or faces, we have covered all the possibilities. (Computing the dual entity for each case is an elementary exercise.) More precisely, we first compute the conjugate entities for all the primal edges. Then we recover dual entities associated with primal faces and vertices as follow.

3.4.1 Dual entities coming from primal faces

Assume F is a primal face. If F has more than two edges, we can recover the dual entity $F^* = \nabla f(F)$ of F using the conjugate entities associated with the two (or more) primal edges that bound F . Indeed, we have

$$F^* = \text{co}\{\nabla f(x_i) : F = \text{co}\{x_i\}\}$$

where x_i are the extreme points of F (the formula extends to the case F is unbounded by considering points at infinity).

The only remaining cases are when F has no edge (treated as a special case in Step 1 of the algorithm), or when F has a single edge. In the later case, one of the following cases occur:

- the edge is a linear edge. Using the linear edge tree, either the neighbor face is unbounded (and the conjugate entity is a ray), or we have a f_i/f_j case, which we further subdivide in a QL / QL case, a LL / QL case, a LL / LL case, and a QL / LL case (since the edge is linear we cannot have a QQ face). The conjugate entity of the linear edge allows us to compute the conjugate by applying the linear edge tree except for the f_i/f_j smooth with QL / QL faces case (the conjugate of the edge is a vertex and we fail to obtain the two rays from the primal edge information). Hence, a special case is introduced to deal with a line separating two QL faces.
- the edge is a quadratic edge. The quadratic edge tree captures all the possibilities i.e. the conjugate entity of the edge allows us to recover the full conjugate.

We now deal with conjugate vertices.

3.4.2 Dual entities coming from primal vertices

Assume V is a primal vertex.

If $V \in \text{intdom } f$ the primal edges incident to V capture enough information to recover

$$V^* = \{\text{co}\nabla f_i(x_i) : x_i \in F_i \text{ and } \cap_i F_i = \{V\}\}$$

where x_i is any point in F_i and F_i is a face incident to V . Each face F_i has an edge incident to V , and we recover V^* using the conjugate of these edges.

If $V \in \text{bddom } f$ and $\text{intdom } f$ is nonempty, the conjugate entities of the primal edges incident at V allow us to compute the conjugate, and no special case is needed.

Finally, if $V \in \text{bddom } f$ and $\text{intdom } f$ is empty either V is an isolated point i.e. f is a needle function (that case is handled in Step 1 of the algorithm) or V is the extreme point of an edge. In the later case, the conjugate of that edge allows us to recover V^* .

3.4.3 The complexity of building the dual arrangement

While the previous two sections proved the algorithm is correct, we now focus on the worst-case time complexity of the algorithm.

All computations in the algorithm occur in linear time except when we check for the existence of an edge or a vertex in the dual arrangement. Consider a primal arrangement with a vertex v_p of degree $O(n)$. The first time an edge e_p connected to that vertex is considered, we create part of its dual entity and the dual entity v_p^* of v_p . Then every time an edge connected to v_p is considered we need to detect that v_p^* has already been created and retrieve its index in the dual arrangement.

Similarly, if we have an LL face F_p with $O(n)$ edges in the primal arrangement, we first create its conjugate entity (a vertex F_p^*) in the dual arrangement. Then for every edge bounding the face F_p , we check that F_p^* exists and retrieve its index.

Another case to be considered is when we have a QL face with two parallel quadratic edges. Then the dual entity associated with the face is a single quadratic edge, and we need to detect whether it has already been created.

Eventually, we need to create a dual vertex only once to respect the integrity of the arrangement. Thus for each gradient value we need to check whether the associated vertex has been created before. If it has been not been created, create it and record its index in the dual arrangement. Otherwise, do not recreate it, just use its index. Consider again the convex PLQ function of Fig. 1. Figure 7 illustrates the steps of the algorithm. After the vertices and edges are created, we add the faces. Then we loop through each entity to compute its associated function value.

Our current implementation loops through every gradient associated with a primal edge, creates its dual vertex (getting a vertex index), and associates the dual vertex index to all the gradients connected with the current primal edge (gradients with primal edges that share a (primal) vertex with the current primal edge). We look at all the gradients on edges sharing a vertex with the current edge and if they are equal (up to floating point error) to the current gradient we directly record within their data structure the index of the dual vertex. Consequently, either this is the first time we consider a gradient and we create its dual vertex, or we already considered that gradient before and recorded the dual vertex index. Hence, we never create duplicate vertices. Similarly we make sure we never create a duplicate edge.

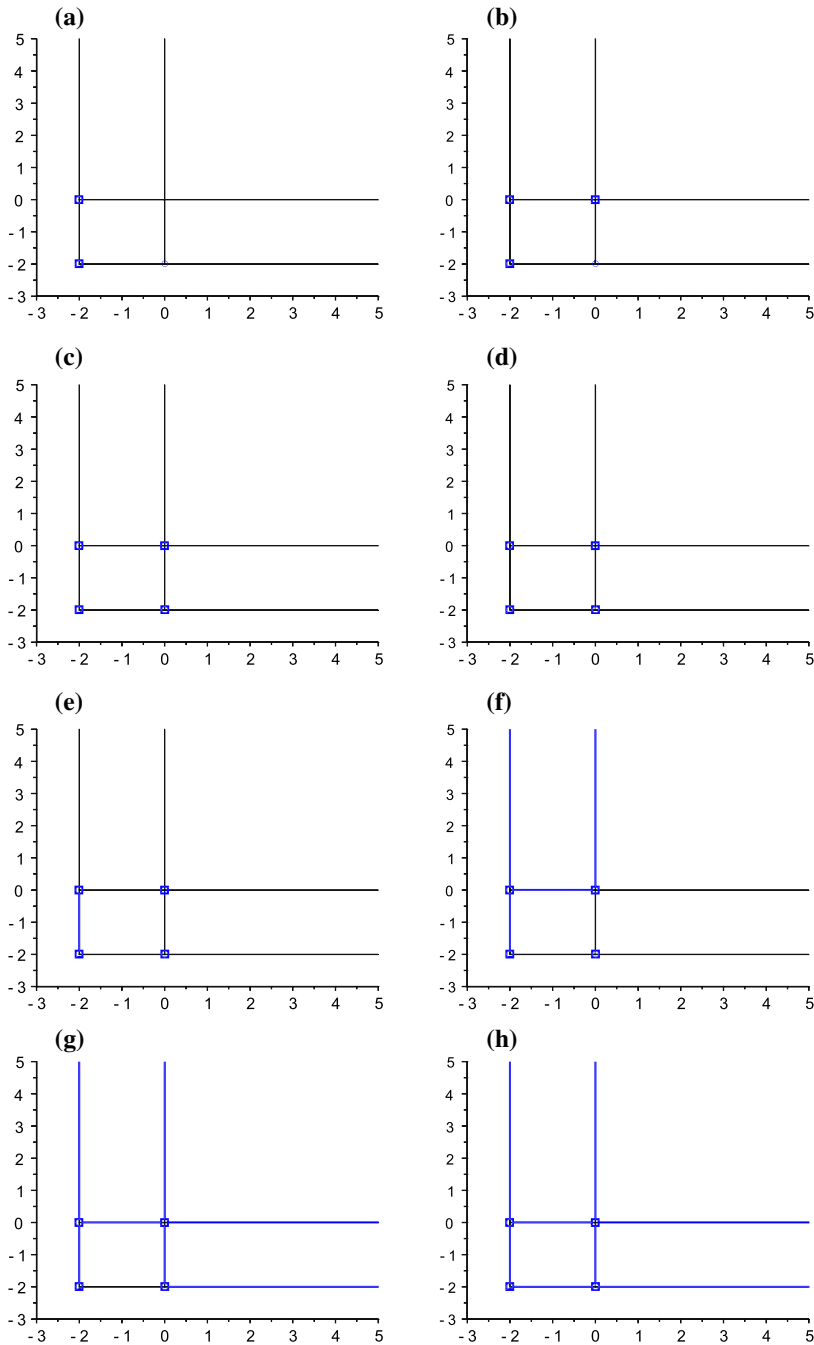


Fig. 7 The 8 steps to create the arrangement of the conjugate of the function defined in Fig. 1. Using a propagation front we only create each entity once. **a** Create vertices for e_1^* . **b** Create vertices for e_2^* . **c** Create vertices for e_3^* . **d** Create vertices for e_4^* . **e** Create edges for e_1^* . **f** Create edges for e_2^* . **g** Create edges for e_3^* . **h** Create edges for e_4^*

However, when the primal arrangement has a vertex connected to $O(n)$ edges (vertex of degree $O(n)$) then to build the dual arrangement we need to propagate each conjugate edge E^* to the conjugate of all the (primal) edges adjacent to the primal edge E . (Adjacent edges are all edges that share a vertex with the primal edge.) In that case there are $O(n)$ of them. Hence, the first time we propagate to n edges, the second time to $n - 1$, etc. A linear search gives a quadratic complexity for the algorithm.

In the special case when the primal arrangement has vertices of bounded degree and faces with a bounded number of edges (e.g. when the primal arrangement is a grid), we obtain a linear worst-case running time. To handle the general case we turn to more advanced data structures.

The problem amounts to considering a finite set of n points in the plane and to assigning a unique index to each point. In our case the points are the gradients at the endpoints of the primary edges. (We proceed similarly to make sure no duplicate edge is created.) Detecting duplicate points is the key to an efficient algorithm.

Using a binary search tree results in a $O(n \log n)$ complexity (the binary tree uses lexicographic order to keep the gradients sorted and each time a gradient is considered it searches the tree to decide whether to insert a new gradient or find the associated dual vertex). The running time is the same as sorting all the gradients by lexicographic order and then assigning an index. Building the binary search tree takes $O(n \log n)$ time and looking up one gradient in it takes $O(\log n)$ for a total *worst-case* running time of $O(n \log n)$.

Since we do not need to sort but only require to detect duplicate values, we can use a hash table to store the gradients. The worst case complexity of building the hash table is superlinear [13] but its expected building time is linear using a minimal perfect hash function [8]. The look-up time is then $O(1)$ (constant) for a total *expected* running time of $O(n)$.

To improve on the worst-case running time, we can use a compressed binary trie (also called a radix tree and described in general computer science textbooks on data structures like [18]). A compressed trie uses (optimal) $O(n)$ space, can be built in $O(mn)$ time, and performs look-up queries in $O(m)$ time, where m is the size of the key. In our case, using single precision floating point numbers in the IEEE 754 format we can build a 64 bit key by putting the 32 bits representation of each coordinate of the point together, i.e., if $P = (x, y)$ we build a key $k = b_x b_y$ where b_x (resp. b_y) is the binary representation of x (resp. y). Assuming m is bounded (like we did for the running time of the binary search tree and the hash table) we obtain a worst-case linear running time.

In practice, when we compute the conjugate to plot it we typically consider primal arrangements with at most 10,000 entities, e.g. a 100×100 grid. We can reduce the size of each single floating point number to 16 bits by using a precision of 10^{-2} (sufficient for plotting), i.e., we drop all the lowest bits i with $2^{-i} < 10^{-2}$. Since we are comparing gradients we put the two 16 bits numbers together to obtain a 32 bit key. Assuming multiplicative constants for all three data structures are within the same range, the binary search tree requires around $32 * 10,000 \log_2(10,000) \approx 4,250,000$ operations (only $10,000 \log_2(10,000) \approx 133,000$ if we assume comparing numbers takes $O(1)$ time). The expected number of operations for the hash table is $32 * 10,000 = 320,000$. For the compressed trie we obtain around 320,000 as a worst-case running time.

Consequently, the algorithm runs in linear worst-case time when implemented using a compressed trie data structure.

4 Examples

The algorithm is implemented in Scilab [32] using the computational geometry library CGAL [10] through the gateway library CGLAB [11].

Remark 4 Most of our examples use functions of the form $f(x_1, x_2) = f_1(x_1) + f_2(x_2)$ i.e. separable functions to allow us to check the computation with univariate PLQ algorithms [16, 27]. More general examples can be generated by pre-composing such function by a rotation to obtain a nonzero coefficient for the $x_1 x_2$ term. However, the conjugation operation is invariant by rotation in the sense that the function $f(x) = g(Rx)$, with R a rotation matrix and g a proper lsc convex function, admits for conjugate $f^*(s) = g^*(Rs)$. (Apply [31, Theorem 11.23(b)] and use $R^{-1} = R^T$.) Consequently not much more generality is gained by pre-composing by a rotation.

We now illustrate the computation on several examples. Consider the bivariate function

$$f(x_1, x_2) = \begin{cases} x_1^2 - 1 & \text{if } |x_1| \geq 1 \text{ and } x_2 = 0, \\ 0 & \text{if } |x_1| \leq 1 \text{ and } x_2 = 0, \\ +\infty & \text{otherwise.} \end{cases}$$

i.e. f is the convex envelope of $|x_1^2 - 1|$ plus the indicator of the x -axis. Then the conjugate of f can be computed using our algorithm for convex PLQ bivariate functions or one of the algorithms for univariate convex PLQ functions [16, 27]. Figure 8 shows that the results agree.

Consider the function

$$f(x_1, x_2) = \begin{cases} 0 & \text{if } x_1, x_2 \leq 0, \\ x_2 - \frac{1}{4} & \text{if } x_2 \geq x_1 \text{ and } x_2 \geq \frac{1}{2}, \\ x_2^2 & \text{if } x_2 \geq x_1 \text{ and } x_2 \in [0, \frac{1}{2}], \\ x_1^2 & \text{if } x_2 \leq x_1 \text{ and } x_1 \in [0, \frac{1}{2}], \\ x_1 - \frac{1}{4} & \text{if } x_2 \leq x_1 \text{ and } x_1 \geq \frac{1}{2}. \end{cases} \quad (7)$$

It illustrates the case of a quadratic segment between $(0, 0)$ and $(1/2, 1/2)$ with $A \wedge B'$, since the gradients of the two QL faces coincide with the value $(0, 0)$ at $(0, 0)$, while the gradients of the QL faces differ at every other point along the line segment (see Fig. 9).

Using Formula (1) we can compute the Moreau envelope of a convex PLQ bivariate function using the following operations: compute $\|\cdot\|^2/2 + \lambda f$ by multiplying all the quadratic forms by λ and add half the identity matrix (keep the same planar arrangement), compute the conjugate of that function, multiply all the resulting quadratic forms by $-1/\lambda$ and add $1/(2\lambda)$ times the identity matrix. The computation

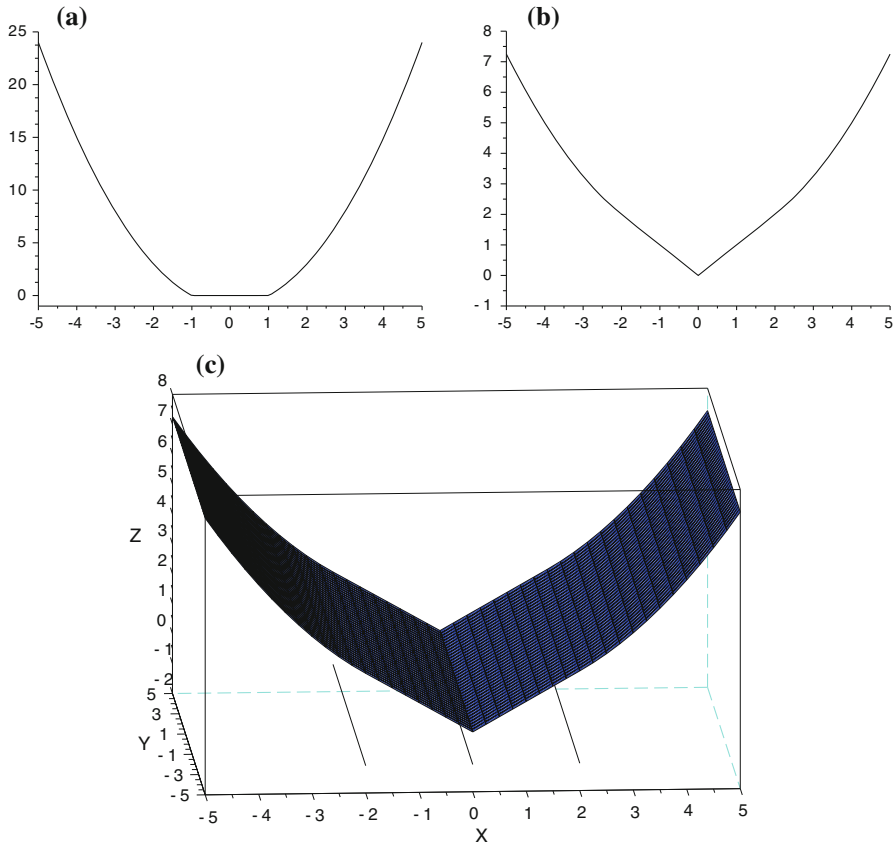


Fig. 8 Computing the conjugate of a univariate function f using both the univariate and bivariate convex PLQ algorithms. **a** The original function f , plotted against x_1 , at $x_2 = 0$. **b** The conjugate f^* using the univariate PLQ conjugate algorithm. **c** The full-domain conjugate f^* using the bivariate PLQ conjugate algorithm

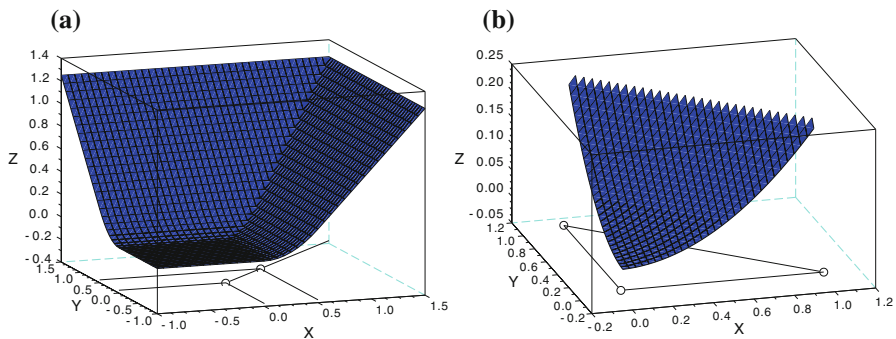


Fig. 9 A function with a quadratic edge segment in the case $A \cap B'$. **a** The (primal) function f from Formula (7). **b** The conjugate f^* has a single QL face resulting from the single quadratic edge in f

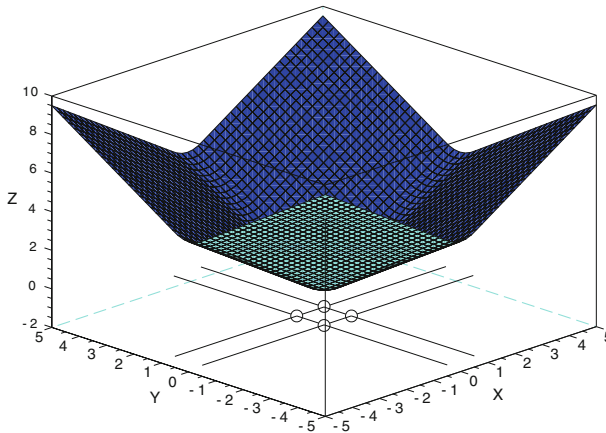


Fig. 10 The Moreau envelope of $f(x_1, x_2) = |x_1| + |x_2|$, with $\lambda = 1/2$

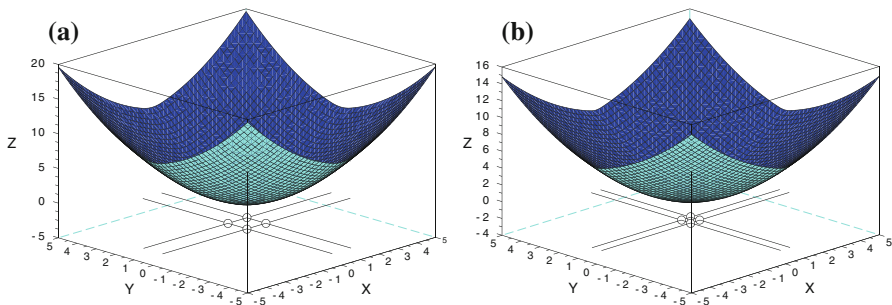


Fig. 11 Smoothing operators for the function $f(x_1, x_2) = |x_1| + |x_2|$, with $\lambda = 1/2$. **a** The smoothing operator $s_\lambda f$. **b** The smoothing operator $T_\lambda f$

is obviously linear time since the only nontrivial operation is the computation of the conjugate. (Note that we do not use the addition algorithm for two convex PLQ functions as it would result in log-linear complexity and is unnecessary here.) We illustrate the computation by computing the Moreau envelope of $f(x_1, x_2) = |x_1| + |x_2|$ in Fig. 10.

In addition, we illustrate the smoothing operators $T_\lambda f$ and $s_\lambda f$ in Fig. 11. Both operators can obviously be computed in linear time.

Finally, Fig. 12 shows the proximal average of $f_1(x_1, x_2) = x_1^2$ on $[-1, 1] \times \{1\}$ with the bounded linear function going through $(-1, -1)$, $(0, -2)$, $(1, -1)$ i.e. with $f_2(x_1, x_2) = x_1 + 1$.

5 A toolbox for convex PLQ bivariate functions

Table 2 summarizes the operators implemented with their worst-case time complexity. The first three operators form the core from which all other operators are composed of. The map overlay problem has a worst-case quadratic complexity when $k = n_1 n_2$, i.e.,

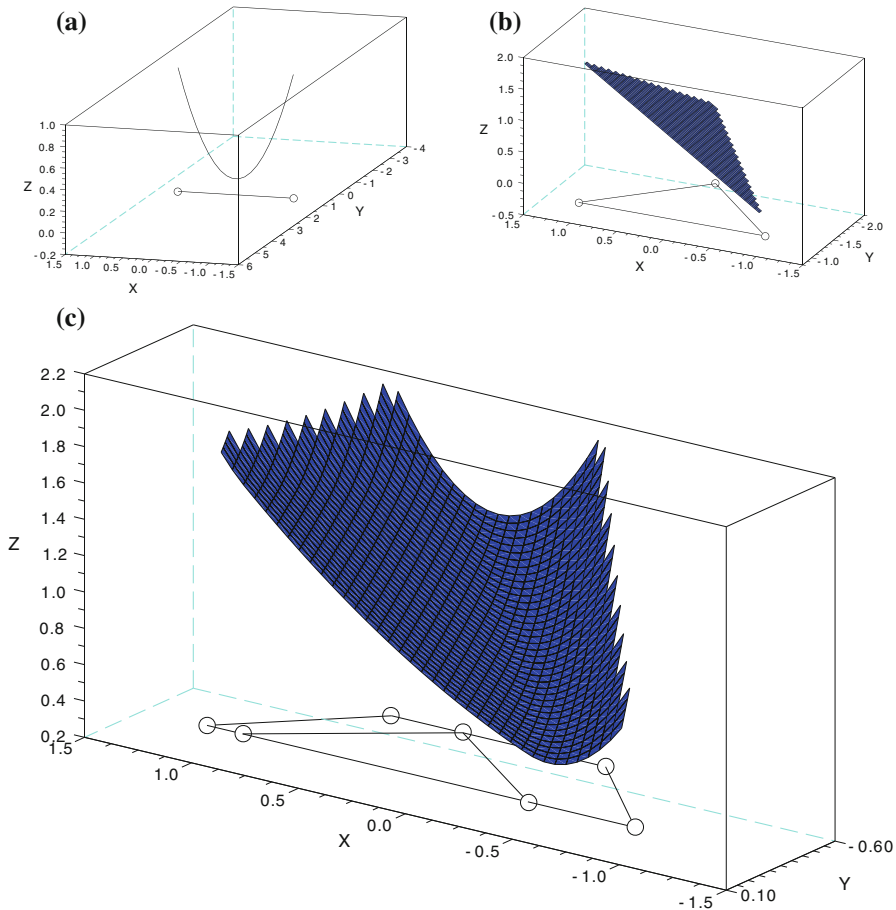


Fig. 12 The proximal average. **a** The function $f_1(x_1, x_2) = x_1^2$ on the line segment between $(-1, 1)$ and $(1, 1)$. **b** The function $f_2(x_1, x_2) = x_1 + 1$ on the triangle defined by $(-1, 1)$, $(0, 2)$, $(1, -1)$. **c** The proximal average of f_1 with f_2 : $P_1(f_1, f_2; 1/2, 1/2)$

when all faces of one function intersect all faces of the other functions [14, Chapter 2]. However, we prefer to list an output sensitive complexity that is much more likely to occur: when the number of output faces is on the same order as the number of input faces the computation can be performed in log-linear time. The point location problem is a classical problem in computational geometry with a worst-case logarithmic query time, see [14, Chapter 6] and references therein.

The resulting toolbox is extremely efficient as no algorithm requires more than log-linear time (except in extreme cases for the addition). Interestingly the most costly operation is the addition. Note also that the complexity results differ from the one dimensional case for three operators: addition, proximal average (because it is based on the addition), and the maximum (which is not always a PLQ function for bivariate functions).

Table 2 Core operators of a toolbox for convex PLQ bivariate functions and their complexity

Operation	Complexity 2D	Complexity 1D	Problem
Conjugate	$O(n)$	$O(n)$	
Addition	$O((n_1 + n_2 + k) \log(n_1 + n_2))$	$O(n_1 + n_2)$	Map overlay
Pointwise evaluation	$O(\log n)$	$O(\log n)$	Point location
Grid evaluation	$O(n + M)$	$O(n + M)$	
Scalar multiplication	$O(n)$	$O(n)$	
Moreau envelope	$O(n)$	$O(n)$	
Proximal average	$O((n_1 + n_2 + k) \log(n_1 + n_2))$	$O(n_1 + n_2)$	
Maximum	Not a PLQ function	$O(n)$	
Convex envelope	Unknown	$O(n)$	

The input functions f (resp. f_1, f_2) have n (resp. n_1, n_2) entities. The addition operator is output sensitive: k is the sum of all entities of the resulting planar arrangement ($k = n_1 n_2$ in the worst case). The grid evaluation complexity relates to a $m \times m$ grid with $M = m^2$ points. All algorithms achieve their worst-case time complexity

Remark 5 Convergence results of the type “assuming p is a proper lsc convex PLQ function close to f a lsc proper convex function then p^* converges to f^* ” have been proven in the framework of the fast algorithms [12, 21]. Briefly, the smoother the function, the faster the convergence. These results apply immediately to our framework by considering a piecewise linear approximation p of f e.g. take p as the linear spline interpolating f on a grid that becomes denser and denser.

Note that one does not need to use the “quadratic part” of p for convergence results. The rationale for manipulating PLQ functions instead of piecewise linear functions is to still have a PLQ function after taking the Moreau envelope.

6 Conclusion

We presented the first algorithm to compute numerically the conjugate of convex PLQ bivariate functions. The algorithm is an hybrid symbolic-numeric algorithm that uses computational geometry data structures and optimal algorithms to achieve an optimal linear time complexity. By adding the addition and scalar multiplication operators, we obtained the core operations forming a toolbox for convex bivariate PLQ functions. We illustrated its use by computing the Moreau envelope, a couple of regularization operators proposed by R. Goebel and S. Moffat, and the proximal average.

The toolbox can be extended to nonconvex operators by implementing an algorithm to compute the convex envelope of nonconvex PLQ bivariate functions provided the result is still a PLQ function. The other operators to be implemented are all the partial operators e.g. taking the partial conjugate with respect to only one variable. Both of these tasks are under investigation.

Acknowledgments The authors would like to thank an anonymous referee for pointing out reference [34], constructive comments that improved the presentation of the algorithm, and pointing a special configuration that resulted in adding Sect. 3.4 to explain the complexity of the algorithm.

References

1. Bauschke, H.H., Goebel, R., Lucet, Y., Wang, X.: The proximal average: basic theory. *SIAM J. Optim.* **19**(2), 768–785 (2008). doi:[10.1137/070687542](https://doi.org/10.1137/070687542)
2. Bauschke, H.H., Lucet, Y., Trienis, M.: How to transform one convex function continuously into another. *SIAM Rev.* **50**(1), 115–132 (2008). URL: <http://link.aip.org/link/?SIR/50/115/1>
3. Bauschke, H.H., Lucet, Y., Wang, X.: Primal-dual symmetric intrinsic methods for finding antiderivatives of cyclically monotone operators. *SIAM J. Control Optim.* **46**(6), 2031–2051 (2007). URL: <http://link.aip.org/link/?SJC/46/2031/1>
4. Bauschke, H.H., Matoušková, E., Reich, S.: Projection and proximal point methods: Convergence results and counterexamples. *Nonlinear Anal.* **56**(5), 715–738 (2004). doi:[10.1016/j.na.2003.10.010](https://doi.org/10.1016/j.na.2003.10.010)
5. Bauschke, H.H., Moffat, S.M., Wang, X.: Self-dual smooth approximations of convex functions via the proximal average. In: Bauschke, H.H., Burachik, R.S., Combettes, P.L., Elser, V., Luke, D.R., Wolkowicz, H. (eds.) *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*, Springer Optimization and Its Applications, vol. **49**, pp. 23–32. Springer, New York (2011). doi:[10.1007/978-1-4419-9569-8_2](https://doi.org/10.1007/978-1-4419-9569-8_2)
6. Bauschke, H.H., Mohrenschildt, M.V.: Symbolic computation of Fenchel conjugates. *ACM Commun. Comput. Algebra* **40**(1), 18–28 (2006). doi:[10.1145/1151446.1151453](https://doi.org/10.1145/1151446.1151453)
7. Borwein, J.M., Hamilton, C.H.: Symbolic Fenchel conjugation. *Math. Program.* **116**(1), 17–35 (2008). doi:[10.1007/s10107-007-0134-4](https://doi.org/10.1007/s10107-007-0134-4)
8. Botelho, F.C., Lacerda, A., Menezes, G.V., Ziviani, N.: Minimal perfect hashing: a competitive method for indexing internal memory. *Inf. Sci.* **181**(13), 2608–2625 (2011). URL: <http://www.sciencedirect.com/science/article/pii/S0020025509005271>
9. Brenier, Y.: Un algorithme rapide pour le calcul de transformées de Legendre-Fenchel discrètes. *C. R. Acad. Sci. Paris Sér. I Math.* **308**, 587–589 (1989)
10. CGAL, Computational Geometry Algorithms Library. URL: <http://www.cgal.org>
11. CGLAB a Scilab toolbox for geometry based on CGAL. URL: <http://cglab.gforge.inria.fr/>
12. Corrias, L.: Fast Legendre-Fenchel transform and applications to Hamilton-Jacobi equations and conservation laws. *SIAM J. Numer. Anal.* **33**(4), 1534–1558 (1996). URL: <http://www.jstor.org/stable/2158316>
13. Czech, Z.J., Havas, G., Majewski, B.S.: Perfect hashing. *Theor. Comput. Sci.* **182**(1–2), 1–143 (1997). URL: <http://www.sciencedirect.com/science/article/pii/S0304397596001466>
14. de Berg, M., van Kreveld, M., Overmars, M., Schwarzkopf, O.: *Computational Geometry*, 3rd edn. Springer, Berlin (2008). URL: <http://www.springer.com/computer/theoretical+computer+science/book/978-3-540-77973-5>. Algorithms and applications
15. Gardiner, B., Lucet, Y.: Convex hull algorithms for piecewise linear-quadratic functions in computational convex analysis. *Set-Valued Var. Anal.* **18**(3–4), 467–482 (2010). doi:[10.1007/s11228-010-0157-5](https://doi.org/10.1007/s11228-010-0157-5)
16. Gardiner, B., Lucet, Y.: Graph-matrix calculus for computational convex analysis. In: Bauschke, H.H., Burachik, R.S., Combettes, P.L., Elser, V., Luke, D.R., Wolkowicz, H. (eds.) *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*, Springer Optimization and Its Applications, vol. **49**, pp. 243–259. Springer, New York (2011). doi:[10.1007/978-1-4419-9569-8_12](https://doi.org/10.1007/978-1-4419-9569-8_12)
17. Goebel, R.: Self-dual smoothing of convex and saddle functions. *J. Convex Anal.* **15**(1), 179–190 (2008). URL: <http://www.heldermann.de/JCA/JCA15/JCA151/jca15012.htm>
18. Goodrich, M.T., Tamassia, R.: *Data Structures and Algorithms in Java*, 5th edn. Wiley, New York (2010)
19. Hiriart-Urruty, J.B., Lemaréchal, C.: *Convex Analysis and Minimization Algorithms*, Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences], vol. **305–306**. Springer, Berlin (1993). URL: <http://www.springer.com/math/book/978-3-540-56850-6>. Vol I: Fundamentals, Vol II: Advanced theory and bundle methods
20. Johnstone, J., Koch, V., Lucet, Y.: Convexity of the proximal average. *J. Optim. Theory Appl.* **148**, 107–124 (2011). doi:[10.1007/s10957-010-9747-5](https://doi.org/10.1007/s10957-010-9747-5)
21. Lucet, Y.: A fast computational algorithm for the Legendre-Fenchel transform. *Comput. Optim. Appl.* **6**(1), 27–57 (1996). URL: <http://www.springerlink.com/content/x777265871618672/>
22. Lucet, Y.: Computational Convex Analysis Library (1996–2011). URL: <https://people.ok.ubc.ca/ylucet/ccal.html>

23. Lucet, Y.: Faster than the Fast Legendre Transform, the Linear-time Legendre Transform. *Numer. Algorithms* **16**(2), 171–185 (1997). URL: <http://www.springerlink.com/content/m41t352758814q50/>
24. Lucet, Y.: A linear Euclidean distance transform algorithm based on the Linear-time Legendre Transform. In: *Proceedings of the Second Canadian Conference on Computer and Robot Vision (CRV 2005)*, pp. 262–267. IEEE Computer Society Press, Victoria BC (2005)
25. Lucet, Y.: Fast Moreau envelope computation I: numerical algorithms. *Numer. Algorithms* **43**(3), 235–249 (2006). doi:[10.1007/s11075-006-9056-0](https://doi.org/10.1007/s11075-006-9056-0). URL: <http://www.springerlink.com/content/h80k45x24t1q7426/>
26. Lucet, Y.: What shape is your conjugate? A survey of computational convex analysis and its applications. *SIAM Rev.* **52**(3), 505–542 (2010). doi:[10.1137/100788458](https://doi.org/10.1137/100788458)
27. Lucet, Y., Bauschke, H.H., Trienis, M.: The piecewise linear-quadratic model for computational convex analysis. *Comput. Optim. Appl.* **43**, 95–118 (2009). URL: <http://www.springerlink.com/content/j726881521t4541/>
28. Moffat, S.M.: On the Kernel Average of n Functions. Master's thesis, Department of Mathematics, University of British Columbia (2009). URL: <http://hdl.handle.net/2429/21932>
29. Moreau, J.J.: Proximité et dualité dans un espace Hilbertien. *Bull. Soc. Math. France* **93**, 273–299 (1965). URL: http://www.numdam.org/item?id=BSMF_1965__93__273_0
30. Noullez, A., Vergassola, M.: A fast Legendre transform algorithm and applications to the adhesion model. *J. Sci. Comput.* **9**(3), 259–281 (1994). URL: <http://www.springerlink.com/content/h8733538m6523g82>
31. Rockafellar, R.T., Wets, R.J.B.: *Variational Analysis*. Springer, Berlin (1998). URL: <http://www.springer.com/math/book/978-3-540-62772-2>
32. Scilab Consortium: *Scilab* (1994). URL: <http://www.scilab.org>
33. She, Z.S., Aurell, E., Frisch, U.: The inviscid Burgers equation with initial data of Brownian type. *Commun. Math. Phys.* **148**(3), 623–641 (1992). URL: <http://projecteuclid.org/euclid.cmp/1104251047>
34. Sun, J.: On the structure of convex piecewise quadratic functions. *J. Optim. Theory Appl.* **72**(3), 499–510 (1992). doi:[10.1007/BF00939839](https://doi.org/10.1007/BF00939839)